FULL-STACK AI SOFTWARE ENGINEERING PROJECT

AI RESUME REVIEWER & ATS INTELLIGENCE

Comprehensive Technology Stack Inventory, Architectural Blueprint, Scoring Algorithms, and Slide-by-Slide Presentation Defense Manual.

React 19 (Vite 6) | Node.js & Express | MongoDB & Mongoose | Google Gemini 2.0 Flash | ATS Heuristic Engine | Job Match Scorer | JWT & Google OAuth 2.0 | Playwright Browser E2E

PROJECT ARCHITECTURE
Decoupled Client-Server REST Microservices

AI MODEL INTEGRATION
Google GenAI SDK (Gemini 2.0 / 1.5 Flash)

DATABASE ARCHITECTURE
MongoDB Atlas / Mongoose + In-Memory Fallback

DOCUMENT PARSERS
PDF (pdf-parse) & Word DOCX (Mammoth)

PRESENTATION FOCUS
Viva Defense, Architecture Review, Tech Stack

SECURITY SUITE
Bcrypt.js, JWT, Sanitization & Guest Isolation

1 Executive Summary & System Overview

The **AI Resume Reviewer** is an enterprise-grade full-stack platform engineered to solve the "black box" problem of modern recruitment. Over **75% of resumes** submitted online are filtered out before reaching a human recruiter due to automated *Applicant Tracking Systems (ATS)* parsing errors, missing technical keywords, poor typographic flow, and unquantified job bullets.

📄 Ingestion Engine

Supports multi-format binary uploads (.PDF and .DOCX) with automated buffer parsing, optical layer scanning, and structured section extraction.

🎯 Heuristic & AI Audit

Dual-layer scoring: deterministic keyword boundary matching + Google Gemini 2.0 Flash for semantic feedback and bullet rewriting.

⚡ Zero-Friction UX

Immediate guest session mode, side-by-side resume version diffing, interactive score badges, and offline fallback resilience.

2 High-Level Architecture & Data Flow



> **Core Architectural Strength:** The system is designed with *fail-safe autonomy*. If MongoDB is offline, an in-memory replica activates seamlessly. If the Google Gemini API key is absent or exhausted, deterministic heuristics calculate scores without disruption.

3 Complete Technology Stack Inventory

Every tool, library, framework, and language chosen for this project serves a precise engineering rationale:

Layer / Domain	Technology & Version	Engineering Rationale & Practical Purpose
Frontend Core	`React 19.0.0`	Latest stable React release utilizing concurrent rendering, modern hooks, fine-grained virtual DOM reconciliation, and fluid state updates.
Build Tool & Bundler	`Vite 6.2.0`	Next-gen build tool replacing Webpack. Native ES-modules provide sub-second hot module replacement (HMR) and optimized Rollup production bundles.
Client Routing	`React Router v7.2.0`	Declarative SPA client routing with protected routes, parameter extraction (:resumeId), lazy view transitions, and browser history synchronization.
Iconography	`Lucide React 0.475.0`	Tree-shakeable, lightweight SVG icon package ensuring clean, modern UI visual cues with zero runtime bloat.
Gamification & Effects	`Canvas Confetti 1.9.4`	HTML5 canvas particle physics generator triggering celebratory rewards when candidate achieves exceptional ATS or job match scores.
Design & Styling	`Pure Vanilla CSS3`	Tailored design system with CSS custom properties (tokens.css), glassmorphic translucent layers, flexbox, CSS grid, and dark/light mode themes.
Backend Runtime	`Node.js (v18+ / v24)`	High-throughput, non-blocking asynchronous event loop with native ECMAScript Modules ("type": "module").
Web Framework	`Express.js 4.21.2`	Minimalist, robust HTTP server framework for modular API routing, middleware chaining, and unified JSON error handling.
Database Engine	`MongoDB`	NoSQL document database ideal for unstructured resume JSON schemas, flexible scoring models, and dynamic keyword audits.
Object Data Modeling	`Mongoose 8.9.5`	Provides strict schema typing, pre/post middleware hooks, relational document population, and validation constraints.
PDF Parsing Engine	`pdf-parse 1.1.1`	Direct memory buffer extraction of text streams from uploaded PDF resumes without external binary dependencies.
DOCX Parsing Engine	`Mammoth 1.9.0`	Extracts clean, semantic text and paragraph headings from Microsoft Word .docx documents by parsing underlying OpenXML.
Generative AI Model	`@google/genai 2.28.0`	Google Gemini 2.0 Flash SDK performing semantic evaluation, personalized candidate advice, and Google X-Y-Z bullet rewrites.
Authentication Security	`JWT & Bcrypt.js`	Stateless JSON Web Tokens with 7-day expiration combined with 10-round salted one-way bcrypt password hashing.
Single Sign-On (SSO)	`google-auth-library`	Validates Google OAuth 2.0 cryptographic ID tokens, facilitating seamless 1-click Google authentication.
Email Service	`Nodemailer 10.1.7`	SMTP transactional mailer for sending 6-digit email verification codes, welcome emails, and secure password reset tokens.
File Upload Middleware	`Multer 1.4.5`	Streaming multipart/form-data upload pipeline enforcing 10MB size limits and PDF/DOCX MIME type validation.

Layer / Domain	Technology & Version	Engineering Rationale & Practical Purpose
Security Sanitization	`Custom Sanitize MW`	Custom recursive sanitizer stripping dangerous Mongo query keys ($gt, $where) and XSS tags before hitting controllers.
E2E Browser Testing	`Playwright Core 1.64.1`	Automated Chromium/Edge browser testing orchestrating realistic user journeys: upload, audit, comparison, and authentication.

4 Algorithmic Engines & AI Architecture

1. ATS Compatibility & Keyword Engine

Mechanism: Scans resume text against a proprietary technical dictionary containing 50+ keywords across 6 categories (Languages, Frontend, Backend, Databases, Cloud/DevOps, QA/Agile).

- **Regex Word Boundaries:** Employs `\b{keyword}\b` regex matching to eliminate false positives (e.g., "Go" vs "Good").
- **Classification:** Segregates matched skills from missing critical requirements with priority tiers (Critical, High, Medium).
- **Layout Compliance:** Checks single-column linearity, optical extractability, and standard section title naming conventions.

2. Job Description Match Engine

Mechanism: Evaluates role fitness against user-entered job specs or dynamically pasted job descriptions.

- **Role Fit Matrix:** Multi-dimensional scoring across Frontend, Backend, Cloud Infrastructure, Scalability, and Leadership.
- **Seniority Audit:** Cross-references candidate years of experience and title progression against job requirements.
- **Missing Skills Roadmap:** Highlights specific gaps with actionable recommendations for candidate upskilling.

3. Google Gemini 2.0 Flash Integration

Mechanism: Uses Google's next-generation LLM via the official `@google/genai` SDK for deep semantic critique.

- **Strict Schema Enforcement:** System prompts mandate pure JSON conforming to an explicit response structure.
- **Structured Verdicts:** Generates executive headlines, targeted strengths, and high-impact weaknesses.
- **Automatic Fallback:** If Gemini 2.0 Flash is busy, it auto-retries on Gemini 1.5 Flash before invoking deterministic heuristics.

4. Google X-Y-Z Bullet Optimizer

Formula: *"Accomplished [X] as measured by [Y], by doing [Z]"*

- **Weak Bullet Point:** "Handled backend APIs for the payment team."
- **Engine Optimized Bullet:** "Architected resilient payment microservices, reducing API checkout latency by 34% through Redis caching and PostgreSQL query optimization."
- **Rationale:** Embeds measurable KPIs, active verbs, and technical stack ownership into every bullet.

5 Composite Scoring Mathematical Formula

When evaluated against a target Job Description:

```
Overall Score = (ATS_Score × 0.35) + (JobMatch_Score × 0.35) + (Readability × 0.15) + (Impact_Score × 0.15)
```

When evaluated in "Resume Only" standalone mode (no target job specified):

```
Overall Score = (ATS_Score × 0.70) + (Readability × 0.15) + (Impact_Score × 0.15)
```

* Readability assesses word count density (optimal: 300–800 words). Impact score rewards quantified percentage/dollar metrics (numbers with % or $).

6 Slide-by-Slide Presentation Guide (10-Slide Deck)

Use this structured outline to build your presentation slides and deliver an authoritative talk:

Slide 1: Project Title & Mission Overview

SLIDE 1

Visual: Project logo, platform screenshot showing the dashboard, presenter details.

- **Project Name:** AI Resume Reviewer & ATS Intelligence Platform.
- **Objective:** Bridge the communication gap between candidates and modern automated ATS screening software.
- **Core Capability:** Instant multi-page resume parsing, ATS heuristic audits, and AI-powered bullet optimization.

> ***Speaker Notes:*** *"Respected panel, today I am presenting the AI Resume Reviewer. While thousands of candidates apply for jobs daily, over 75% are eliminated by automated filters. Our platform acts as a candidate's personal AI recruiter, highlighting exact flaws before they press submit."*

Slide 2: The Problem — The ATS Black Box

SLIDE 2

Visual: Infographic showing 100 resumes filtering down to 5 due to formatting and keyword drops.

- **Parsing Failures:** Complex multi-column tables and non-standard fonts scramble text in legacy ATS software.
- **Missing Keyword Density:** Candidates fail to match the precise vocabulary required by hiring managers.
- **Unquantified Accomplishments:** Passive responsibility lists fail to stand out during human 6-second recruiter scans.

> ***Speaker Notes:*** *"The primary challenge candidates face isn't lack of skill—it is lack of visibility. Resumes fail simple optical parsing checks or lack quantified achievements."*

Slide 3: System Architecture & Technology Stack

SLIDE 3

Visual: 3-tier architectural diagram (React 19 Frontend -> Express REST API -> MongoDB & Google Gemini AI).

- **Frontend:** React 19 + Vite 6 + Context API + Custom Glassmorphism CSS Design Tokens.
- **Backend:** Node.js + Express.js modular microservice routers + Multer file streaming.
- **Data & AI:** MongoDB/Mongoose + Google GenAI SDK (Gemini 2.0 Flash) + In-Memory Fallback.

> ***Speaker Notes:*** *"We built our application on modern React 19 with Vite for ultra-fast performance, backed by a Node.js REST server and Google's latest Gemini 2.0 Flash model."*

Slide 4: Document Ingestion & Parsing Pipeline

SLIDE 4

Visual: Flow diagram showing file upload (.pdf / .docx) converted into structured JSON sections.

- **Binary Extraction:** pdf-parse extracts clean text streams; mammoth converts Word XML blocks.
- **Regex Segmentation:** Intelligent boundary scanning categorizes Summary, Experience, Education, Skills, and Projects.
- **Contact Scraper:** Validates email, telephone, LinkedIn, and GitHub links via whole-pattern regular expressions.

> ***Speaker Notes:*** *"When a user uploads a document, our backend parses raw buffers into normalized JSON segments without relying on heavy external software."*

6 Slide-by-Slide Presentation Guide (Slides 5–10)

Slide 5: ATS Compatibility & Keyword Density Engine

SLIDE 5

Visual: Screenshot of ATS Analysis page with green "Matched" and orange "Missing" keyword chips.

- **Curated Dictionary:** Indices 50+ mission-critical technologies across 6 technological domains.
- **Boundary Matching:** Uses whole-word regex to avoid inaccurate substring matches.
- **Format Compliance Checks:** Audits single-column flow, optical text layer clarity, and standard heading labels.

Speaker Notes: *"Our ATS engine gives job seekers an unfair advantage by auditing formatting compliance and identifying high-priority missing keywords."*

Slide 6: Google Gemini AI & Bullet Point Rewriting

SLIDE 6

Visual: Before & After comparison card showing an unoptimized bullet converted via the Google X-Y-Z formula.

- **Google GenAI SDK:** Connected to Gemini 2.0 Flash using strict JSON schema constraints.
- **The X-Y-Z Formula:** Transforms passive duty lists into measurable, business-oriented engineering wins.
- **Resilient Fallback:** If API limits are reached, the system executes built-in heuristic logic seamlessly.

Speaker Notes: *"Recruiters love Google's X-Y-Z formula: Accomplished X, as measured by Y, by doing Z. Our platform automatically rewrites weak bullet points using Gemini 2.0."*

Slide 7: Dynamic Job Description Matcher

SLIDE 7

Visual: Side-by-side view of candidate skills vs. target job criteria with role fit radar/progress bars.

- **Role Fit Breakdown:** Independent scores for Frontend, Backend, Cloud & DevOps, Scalability, and Leadership.
- **On-the-Fly Matching:** Users can paste custom job descriptions for instant role-specific fitness scores.
- **Upskilling Suggestions:** Recommends top 3 technologies to learn for the targeted job.

Speaker Notes: *"Users can benchmark their resume against any specific job post to see their precise fit score before sending an application."*

Slide 8: Authentication, Security & Guest Access

SLIDE 8

Visual: Security architecture diagram showing JWT, Bcrypt, Google OAuth, and input sanitization.

- **Multi-Auth:** Email/Password with bcrypt salt hashing + 1-click Google OAuth 2.0 SSO.
- **Guest Access:** Atomic session generation with concurrency bounds for immediate, barrier-free evaluation.
- **Sanitization Layer:** Custom middleware protects against NoSQL query injections and Cross-Site Scripting (XSS).

Speaker Notes: *"Security is built-in: passwords hashed with bcrypt, stateless JWT tokens, and an isolated guest mode that guarantees zero database bloat."*

Slide 9: Quality Assurance & Automated Testing

SLIDE 9

Visual: Terminal output showing unit test suite and Playwright browser test pass rate.

- **Node.js Native Test Runner:** Executes unit tests and integration tests without bulky third-party runners.
- **Playwright Core E2E:** End-to-end browser automation validating navigation, upload, and score calculations.
- **Concurrency Testing:** Validates atomic guest locks and stress-tests API throughput.

Speaker Notes: *"We verified platform stability using automated Playwright browser tests and Node native unit tests."*

Slide 10: Future Enhancements & Conclusion

SLIDE 10

Visual: Roadmap showing LinkedIn auto-sync, 1-click tailored resume export, and real-time interview prep bot.

- **Roadmap:** 1-Click PDF export with pre-optimized layouts; multi-lingual resume parsing; mock interview AI prep.
- **Summary:** Complete, production-grade full stack system combining deterministic heuristics with generative AI.

Speaker Notes: *"In conclusion, our platform empowers applicants to beat the ATS and get hired. Thank you, and I now welcome your questions."*

7 Technical Viva & Presentation Defense Guide

Prepare for evaluators' toughest technical questions with these verified, authoritative answers:

Q1 Why did you choose React 19 and Vite rather than Next.js or traditional CRA?

Answer: "We selected **React 19 with Vite 6** because our application is a dynamic, authenticated Single Page Application (SPA) dashboard rather than an SEO-reliant e-commerce site. Vite leverages native ES Modules during development, providing instant Hot Module Replacement (HMR) and optimized Rollup bundling that builds in seconds. React 19 provides modern concurrent rendering and refined state synchronization without the deployment overhead of a full Next.js Node server."

Q2 How does the backend parse binary PDF and DOCX files without corrupting characters?

Answer: "We handle file ingestion using **Multer**, which streams files into memory buffers. For PDFs, we utilize **pdf-parse** to decode low-level PDF page streams into UTF-8 text while preserving linebreaks. For Word documents (.docx), we utilize **Mammoth**, which parses the underlying OpenXML format, extracting text and structural headings while stripping complex styling that often confuses ATS parsers."

Q3 Is the ATS score generated purely by AI, or is it algorithmic? Why?

Answer: "The ATS score is calculated using a **hybrid architecture**. Relying solely on LLMs for numerical scores leads to hallucination and non-deterministic variability. Therefore, our ATS scoring uses a **deterministic heuristic engine** with exact regex boundary scans (`\b{keyword}\b`) over 50+ categorized technical terms, single-column linearity checks, and word-density algorithms. Google Gemini AI is then leveraged for what LLMs do best: qualitative semantic critique and creative bullet point rewriting."

Q4 What happens if the Google Gemini API key expires or the user has no internet connection?

Answer: "The architecture incorporates **Graceful Degradation**. In `aiAnalysisEngine.js`, the Gemini call is wrapped in a 15-second timeout with error interception. If the API key is absent or fails, the platform automatically switches to a high-fidelity local heuristic engine that generates comprehensive strengths, weaknesses, and improvement recommendations without throwing errors or breaking the UI."

Q5 How does your backend prevent NoSQL Injection and Cross-Site Scripting (XSS)?

Answer: "We implemented a custom recursive sanitizer middleware (`sanitize.js`) that intercepts every request body, query parameter, and URL parameter. It strips MongoDB operator injection keys starting with `$` or containing `.` (preventing queries like `{ $gt: '' }`) and escapes HTML special entities to neutralize XSS payload injections."

Q6 How does the 'Guest Access' feature work without polluting your MongoDB database?

Answer: "Guest mode creates an atomic session record with a unique guest ID and timestamp in `GuestSession.js`. Guest resumes and analyses are tagged with this ID and subjected to strict server-side concurrency limits. In production, guest sessions are configured with TTL (Time-To-Live) expiration indexes, allowing automatic background cleanup by MongoDB."

Q7 What is Google's X-Y-Z formula, and how does your engine apply it?

Answer: "Google's X-Y-Z formula is: *'Accomplished [X] as measured by [Y], by doing [Z]'*. Our engine inspects candidate bullet points for missing metrics. If a bullet merely lists an action without a measurable outcome, our engine restructures it to incorporate an action verb, a quantified impact percentage or latency metric, and the specific technical stack utilized."